VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI-590015, KARNATAKA

Project Report

on

Cloud-Integrated Robotic Arm for Automated Assembly and Quality Control.

Submitted in partial fulfillment of the requirements for the III semester

Mini Project Work [EC221]

Bachelor of Engineering

in

Electronics and Communication Engineering

of

Visvesvaraya Technological University, Belagavi.

by

Ranjith Kumar TV (1CD23EC133)

Ramya P (1CD23EC131)

Hrithika Sahukar (1CD23EC190)

Yasha AS(1CD23EC188)

Under the Guidance of
Mr. Pavan Kumar E
Manager – R&D Stratergy
External Guide
Elevium – by Nanochip

Under the Guidance of
Prof. Indu

Internal Guide
Dept. of ECE, CIT

Department of Electronics and Communication Engineering
CAMBRIDGE INSTITUTE OF TECHNOLOGY, BANGLORE - 060 036

2024-2025

CAMBRIDGE INSTITUTE OF TECHNOLOGY

K.R. Puram, Bangalore-560 036

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

It is Certified that, **Ms. Hrithika Sahukar** , **Mr. Yashas, Mr. Ranjith Kumar TV** and **Ms. Ramya P bearing USN: 1CD23EC190, 1CD23EC188, 1CD23EC133** and **1CD23EC131** respectively, are bonafide students of Cambridge Institute of Technology, and have completed requirements of the project entitled **"Cloud-Integrated Robotic Arm for Automated Assembly and Quality Control"** in partial fulfillment of the requirements for III semester Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belagavi during academic year 2024-25. It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work Phase-2 prescribed for the Bachelor of Engineering degree.

Mr. Pavan Kumar E
External Guide
Elevium-By Nanochip

Prof Indu
Internal Guide
Dept. of ECE.
CITech

Dr. Shivapanchakshari T. C. Head
of the Dept.
Dept. of ECE. CITech

Dr G. Indhumathi
Principal CITech

External Viva
Name of the Examiners
1.
2.

Signature with date

DECLARATION

We, **Ms. Hrithika Sahukar** , **Mr. Yashas, Mr. Ranjith Kumar TV** and **Ms. Ramya P bearing USN: 1CD23EC190, 1CD23EC188, 1CD23EC133** and **1CD23EC131** respectively, are students of 5TH semester, Electronics and Communication Engineering, Cambridge Institute of Technology, hereby declare that the project entitled **"Cloud-Integrated Robotic Arm for Automated Assembly and Quality Control"** has been carried out by us and submitted in partial fulfillment of the course requirements project work Phase-2 of III semester Bachelor of **Engineering** in **Electronics and Communication Engineering** as prescribed **by Visvesvaraya Technological University, Belagavi,** during the academic year 2024-2025.

We also declare that, to the best of our knowledge and belief, the work reported here does not form part of any other report on the basis of which a degree or award was conferred on an earlier occasion on this by any other student.

Date:

Place: Bengaluru

Ranjith Kumar TV(1CD23EC133)

Hrithika Sahukar(1CD23EC190)

Yashas AS(1CD23EC188)

Ramya P(1CD23EC131)

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to Shri **D. K. MOHAN, Chairman,** Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at CITECH without which this work would not have been possible.

I extremely thankful to Dr. G. **INDUMATHI, PRINCIPAL,** Cambridge Institute of Technology, Bengaluru for providing me Academic ambience and Laboratory facilities to in and everlasting motivation to carry out this work and shaping our careers.

I express my sincere gratitude to **Dr. SHIVAPANCHAKSHARI** T G. HOD, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for his stimulating guidance, continuous encouragement and motivation throughout the course of present work.

I also wish to extend my thanks to our Internal Guide **Prof. Chetan Nair K and Prof. Indu,** Assistant Professor, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for their unstilted support, valuable guidance and help throughout the seminar related work.

I also wish to extend my thanks to **our External Guide Mr. Pavan Kumar E**, Manager-R&D Strategy, Elevium-by Nanochip, Bengaluru for her critical, insightful comments, guidance and constructive suggestion and support to improve the quality of this seminar work.

I would like to thank all the faculty members and non-teaching staff of dept. of ECE and Elevium team for their constant support. And I would like to thank our parents and friends for their constant moral and financial support.

Ranjith Kumar TV(1CD23EC133)
Hrithika Sahukar(1CD23EC190)
Yashas AS(1CD23EC188)
Ramya P(1CD23EC131)

ABSTRACT

This project presents the design and implementation of an intelligent rusted-object detection and pick-and-place system that integrates embedded vision, wireless communication, and robotic manipulation. The system employs two ESP32 microcontrollers, an OV7670 camera module, a 6-DOF robotic arm, and a PCA9685 servo driver to create a compact and efficient automation platform. The first ESP32 functions as the vision node, interfacing with the OV7670 camera to capture live images of objects in the workspace. These frames are transmitted to a personal computer through Wi-Fi, where advanced image processing is performed using the OpenCV library. The PC acts as a lightweight local cloud server, analyzing the streamed images to detect rusted metallic objects based on color segmentation, contour extraction, and pixel-intensity characteristics associated with corrosion.

Once a rusted object is identified, the PC computes its position and transmits control commands to the second ESP32 via Wi-Fi. This controller interprets the received coordinates and actuates the 6-DOF robotic arm through the PCA9685 16-channel PWM driver. The robotic arm executes a programmed pick-and-place sequence, gripping the rusted object and relocating it to a designated zone. Communication between the two ESP32 boards is maintained through UART (RX–TX) for synchronization and status updates.

The system demonstrates a complete closed-loop workflow that links sensing, processing, decision-making, and robotic actuation. It showcases the feasibility of integrating low-cost embedded hardware with a PC-based vision system to automate industrial tasks such as sorting, quality inspection, and material handling. The prototype achieves stable image transmission, reliable object detection, and accurate robotic movement, highlighting its potential as a scalable model for intelligent automation applications.

Contents

1. Chapter 1 : Introduction
2. Chapter 2: Literature Survey
3. Chapter 3: Methodology/Block diagram
4. Chapter 4: Implementation
   Design/Algorithms/Hardware
   & Software used
5. Chapter 5: Programming Code/Algorithm
6. Chapter 6: Simulation/result analysis
7. Chapter 7: Applications

S. Chapter 8: Conclusion and Future scope

9. Chapter 9: Reference

INTRODUCTION

Automation has grown into the backbone of modern industrial operations, where efficiency, consistency, and reliability are prioritized more than ever before. Industries engaged in material handling, quality inspection, maintenance, and sorting continue to adopt systems that combine robotics with intelligent sensing. Among these, vision-based robotic automation has emerged as a powerful tool, enabling machines to perceive their environment, analyze physical conditions, and perform precise tasks without human intervention. Rust detection is one such application where visual inspection plays a fundamental role. Rust formation on metal surfaces not only affects quality but also poses safety risks in manufacturing, storage, and recycling facilities. Manual inspection is slow and inconsistent; therefore, automation becomes essential.

This project focuses on building a compact, cost-effective solution for detecting rusted objects and automatically segregating them using a 6-DOF robotic arm. The system utilizes two ESP32 microcontrollers, the OV7670 camera module, and OpenCV running on a PC acting as a local cloud server. The design demonstrates how affordable embedded hardware and open-source image-processing tools can be integrated to produce an intelligent object-handling system with end-to-end automation capability.

The concept of using dual ESP32 modules enables the separation of computational responsibilities. The first ESP32 is dedicated to vision acquisition. It interfaces with the OV7670 camera sensor, captures real-time images using its digital video interface, and sends the data to a computer through Wi-Fi. The second ESP32 is responsible for robotic control. It interprets commands received from the PC, communicates with the PCA9685 servo driver, and controls the 6-DOF robotic arm required to pick and place the detected rusted object. Communication between the two ESP32 units is handled via UART (RX–TX), ensuring synchronized operation and enabling the robotic controller to stay updated about the camera node's status.

The OV7670 camera module plays a central role in enabling vision for the system. Although it is a low-cost and low-resolution sensor, its output is adequate for detecting rusted regions when proper lighting and preprocessing are used. The frames captured by the module are transmitted to the PC using a lightweight streaming method. The PC processes these frames using the OpenCV library, applying a series of image-processing operations such as color thresholding, morphological filtering, and contour detection. Rust typically exhibits a distinct reddish-brown hue, which allows it to be detected reliably using HSV color segmentation. After identifying the rusted region, the system extracts the object's centroid or bounding-box parameters, which are then translated into positional information for the robotic arm.

Once the object's location is determined, the PC serves as the central decision-making unit—the "cloud" layer of the system. It generates a structured command packet containing details such as object presence, coordinates, and action commands. This packet is transmitted to the robotic ESP32 through Wi-Fi. The robotic controller interprets the packet and maps the object coordinates to corresponding servo angles based on predefined kinematic relationships for the 6-DOF arm. The PCA9685 driver provides precise PWM signals to each servo motor, ensuring smooth and accurate arm movement.

The robotic arm performs a complete pick-and-place operation: moving toward the object, lowering the gripper, closing around the rusted item, and relocating it to a predefined drop zone. This creates a closed-loop cycle where sensing, computation, and mechanical action are seamlessly integrated. The two ESP32 controllers, despite being low-power devices, work collaboratively to realize a system that mirrors real industrial automation practices.

One of the key motivations behind this project is the increasing demand for intelligent robotic systems in manufacturing, metal recycling, warehouse automation, and inspection environments. Rust detection is an important quality-control task, especially in industries where metal components must maintain structural integrity. Implementing a vision-based robotic solution reduces the need for human involvement in repetitive inspection tasks and minimizes error due to fatigue or inconsistent judgment. Furthermore, by using readily available components such as ESP32 boards, a PCA9685 driver, and open-source software like OpenCV, the project highlights how effective automation can be achieved without high-cost industrial equipment.

Another objective of this system is to demonstrate modularity and scalability. The two-ESP32 architecture allows additional modules to be integrated easily—for instance, adding another camera, expanding robotic capabilities, or implementing cloud-based remote control. The PC-based processing provides flexibility, enabling advanced algorithms such as machine learning, template matching, or 3D mapping to be incorporated in the future. Thus, the present system serves not only as a rust-detection platform but also as a foundation for more complex intelligent robotic solutions.

Overall, this project showcases how embedded systems, wireless communication, and computer vision can be integrated into a functional robotic automation system. By combining the strengths of low-cost hardware and powerful vision algorithms, the system delivers accurate rust detection, reliable communication, and smooth robotic operation. The resulting prototype demonstrates the feasibility of deploying intelligent, vision-guided robotic arms in environments where safety, efficiency, and precision are essential.

LITRATURE SURVEY

Vision-guided robotic systems have gained significant attention in recent years due to their capability to automate inspection, sorting, and handling tasks in industrial environments. A combination of camera-based sensing, embedded computing, and robotic manipulators allows real-time decision making and precise actuation, making such systems highly relevant in manufacturing, recycling plants, and quality-control sectors. This literature review examines prior work related to embedded vision systems, rust detection techniques, ESP32-based communication architectures, and servo-driven robotic manipulators to establish the foundation for the present project.

2.1 Embedded Vision Systems

Embedded vision has evolved rapidly with the availability of compact CMOS camera modules such as the OV7670, OV2640, and ArduCam series. Earlier studies highlight the challenges associated with real-time image capture on microcontrollers due to memory constraints and limited processing capabilities. The OV7670, despite being an older low-cost camera, has been widely adopted in educational and research projects for low-resolution image acquisition. Research shows that streaming raw frames to a more powerful processing unit (such as a PC or server) is an efficient strategy to overcome the limitations of microcontrollers. This approach is documented in work involving IoT-based surveillance systems, remote monitoring devices, and low-power machine vision prototypes. These studies align closely with the architecture used in this project, where the ESP32 acts as an image-capture node while outsourcing the heavy image-processing workload to a PC.

2.2 Computer Vision and Rust Detection

Rust detection has been widely studied within industrial inspection systems. Traditional rust identification relies on spectral and color cues, typically appearing in the reddish-brown intensity band. Researchers often use RGB or HSV color-space transformations to isolate rusted regions from metal surfaces. HSV-based thresholding combined with morphological operations such as erosion, dilation, and contour filtering has been proven effective for identifying corrosion in images under controlled lighting conditions.

Advanced approaches include edge detection, texture analysis, and machine-learning-based classification using SVMs or convolutional neural networks. However, many studies reveal that simple threshold-based segmentation remains adequate for low-resolution images and real-time processing environments. This project adopts such classical techniques due to the limited resolution of the OV7670 camera and the requirement for fast, lightweight computation on the PC using OpenCV.

2.3 Wireless Communication in IoT Systems

The ESP32 microcontroller has become a popular choice for IoT applications due to its built-in Wi-Fi, dual-core processor, and support for various communication protocols. Literature

frequently highlights the ESP32's effectiveness for real-time sensor communication, data streaming, and lightweight server-client architectures. Past work demonstrates the suitability of ESP32 for TCP/UDP-based transmission of sensor data, robotics control, and cloud integration.

Research has also shown the value of multi-node communication models, where multiple ESP32 boards collaborate over UART or SPI while transmitting processed data over Wi-Fi to a cloud system. This dual communication approach ensures reliability and modularity—an approach mirrored in the present project, where UART is used for internal synchronization between two ESP32 modules, while Wi-Fi is used for high-level communication with the PC.

2.4 Image Processing Offloading to PCs and Edge Devices

Many recent studies emphasize the benefits of hybrid systems where a microcontroller handles data acquisition while a more powerful device, such as a PC, Raspberry Pi, or cloud server, handles processing. Offloading computation enables higher accuracy in computer vision and allows the use of algorithms that exceed microcontroller capabilities. This architecture is particularly common in mobile robots, remote visual inspection systems, and smart agriculture applications. The present project follows this model by using the PC as a local cloud server running OpenCV for rust detection, ensuring fast and accurate object recognition.

2.5 Robotic Arms and Servo Control Systems

Robotic manipulators, particularly 6-DOF arms, are extensively used in automation for their versatility and ease of control. Literature describes that servo-motor-driven arms provide precise angular control and simpler kinematic modeling compared to stepper systems. The PCA9685 servo controller, widely referenced in robotic projects, is known for delivering stable 12-bit PWM signals and the ability to drive multiple servos simultaneously without burdening the microcontroller's internal timers. Research also shows that ESP32-PCA9685 combinations are commonly used in low-cost robot arms, educational robotic kits, and DIY manipulators due to their reliability and simplicity.

Studies on robotic pick-and-place mechanisms indicate that combining camera-based object detection with servo-driven manipulators significantly enhances automation capabilities. Systems integrating OpenCV with robotic arms commonly rely on coordinate extraction, transformation, and mapping between image space and robotic workspace. Although industrial setups often incorporate advanced robotic kinematics, academic-level systems typically use simplified coordinate mapping and fixed-position pick-and-place routines—an approach consistent with the methodology used in this project.

2.6 Integrated Vision-Guided Robotic Systems

Several research papers explore end-to-end automation pipelines where the sensing module, processing engine, and robotic actuator operate cohesively. These systems often rely on modular architectures enabling scalability, flexibility, and parallel processing. Examples include assembly-line inspection robots, agricultural fruit-picking robots, and recycling robots

capable of sorting materials based on color, texture, or shape. Many such studies emphasize the importance of synchronized communication between camera modules, decision-making processors, and actuators for achieving smooth automation.

The present project fits within this domain by connecting a vision module (ESP32 + OV7670), a processing module (PC with OpenCV), and an actuation module (ESP32 + robotic arm). The literature supports this modular design as an effective approach for building intelligent robotic systems without requiring expensive industrial hardware.

METHODOLOGY

The methodology adopted for this project involves a systematic design approach that integrates hardware components, communication protocols, image-processing techniques, and robotic actuation to achieve rusted-object detection and pick-and-place automation. This section describes the complete workflow, beginning from image acquisition to robotic manipulation, and outlines the step-by-step procedures followed during implementation and testing.

3.1 System Architecture Overview

The overall methodology is structured into three major subsystems:

Vision Acquisition Subsystem – Responsible for capturing images using the OV7670 camera through the first ESP32.

Image Processing and Decision-Making Subsystem – Conducted on the PC acting as a local cloud server using OpenCV.

Robotic Actuation Subsystem – Controlled by the second ESP32 in coordination with the PCA9685 servo driver and 6-DOF robotic arm.

These subsystems communicate with each other via Wi-Fi and UART to complete the sensing–analysis–action loop.

3.2 Hardware Setup and Configuration

3.2.1 OV7670 Camera Interface

The OV7670 module is interfaced with the first ESP32 using:

Pixel data pins (D0–D7)

PCLK for synchronization

VSYNC and HREF for frame timing

SCCB/I2C interface for internal register configuration

The ESP32 initializes the camera by writing configuration registers to set the desired resolution (QQVGA) and color format (RGB565). Due to limited internal RAM, the image is captured line by line and stored in buffers before being transmitted.

3.2.2 Dual ESP32 Communication

Two ESP32 devices are used:

ESP32-Vision Node

ESP32-Robot Controller

The devices communicate through UART (RX–TX) for exchanging status and synchronization signals:

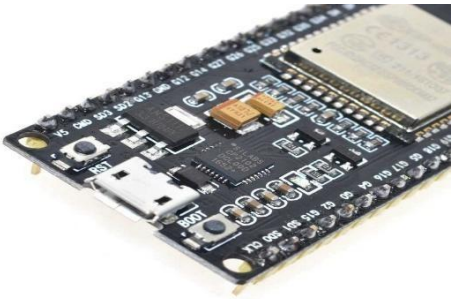
Capture completed

Frame transmission status

Robot operation acknowledgment

Error flags

This ensures coordinated operation between sensing and actuation modules.



3.2.3 PCA9685 Servo Driver Integration

The robotic ESP32 communicates with the PCA9685 via I2C. Methodology steps include:

Initializing PCA9685 registers

Setting PWM frequency (~50 Hz for servos)

Mapping angle values to PWM signals

Testing servo movement for each joint

This ensures smooth and precise control of all 6 servos of the robotic arm.

3.3 Image Capture and Data Transmission

3.3.1 Frame Capture

When triggered, the ESP32-Vision Node performs:

Waits for VSYNC signal

Captures each line of pixel data using PCLK

Stores line segments into buffer

Streams the buffer to the PC in small packets

The camera's low resolution (160×120) helps maintain fast frame capture and low latency.

3.3.2 Wi-Fi Streaming to PC

The ESP32 sets up a Wi-Fi client connection to the PC's TCP/UDP server.

Steps include:

Establishing socket connection

Sending frame metadata (size, format)

Sending pixel data sequentially

Awaiting acknowledgment from PC

This ensures reliable, packet-based frame transmission.

3.4 Image Processing Methodology (OpenCV)

Once the PC receives a full frame, it enters the analysis stage using OpenCV. The methodology includes the following processes:

3.4.1 Preprocessing

Converting the raw buffer into a 2D NumPy array

Applying noise reduction (Gaussian blur)

Converting RGB frame to HSV color space

HSV is preferred due to its robustness under variable lighting.

3.4.2 Rust Detection

Rusted metal is identified using a calibrated HSV threshold range:

Lower threshold: typical brown-orange region

Upper threshold: limits for hue, saturation, and value

A binary mask is generated where rusted pixels are highlighted.

3.4.3 Morphological Operations

To enhance detection accuracy:

Erosion removes small noise spots

Dilation expands significant rust clusters

These operations help isolate genuine rusted regions.

3.4.4 Contour and Coordinate Extraction

Contours are detected on the binary mask. The algorithm selects the largest or most relevant contour to represent the rusted object.

From this contour:

A bounding rectangle is drawn

Centroid coordinates (x, y) are computed

These coordinates are transformed into robot workspace points

If no rust is detected, a flag is sent indicating no action required.

3.5 Decision-Making and Command Generation

Based on the detection results, the PC forms a structured command packet:

```
{
  "object_detected": 1/0,
  "x": <centroid_x>,
  "y": <centroid_y>,
  "action": "pick"
}
```

This packet is transmitted to the robotic ESP32 over Wi-Fi using a TCP client connection. The reliability of the command is ensured using acknowledgment and retransmission mechanisms.

3.6 Robotic Arm Operation

The robotic ESP32 receives the command packet and processes it to carry out the required movement.

3.6.1 Coordinate Mapping

Coordinates from the image frame are mapped to real-world arm coordinates using:

Calibration between camera field-of-view and robotic workspace

Linear or segmented mapping functions

Fixed height/angle adjustments for pick operation

3.6.2 Servo Angle Computation

Basic inverse kinematics or predefined motion sequences are used to:

Rotate the base to align with the object

Adjust shoulder and elbow for reach

Position the wrist for correct orientation

Open/close the gripper

The PCA9685 generates PWM signals to move each servo smoothly.

3.6.3 Pick-and-Place Sequence

The robotic arm performs:

Move to target position

Lower the end-effector

Close gripper to pick rusted object

Lift object

Shift to disposal location

Release object

Return to home position

The controller sends a completion message back to the PC.

Testing and Validation

The system underwent testing under controlled lighting and environmental conditions. Validation steps include:

Testing camera capture reliability

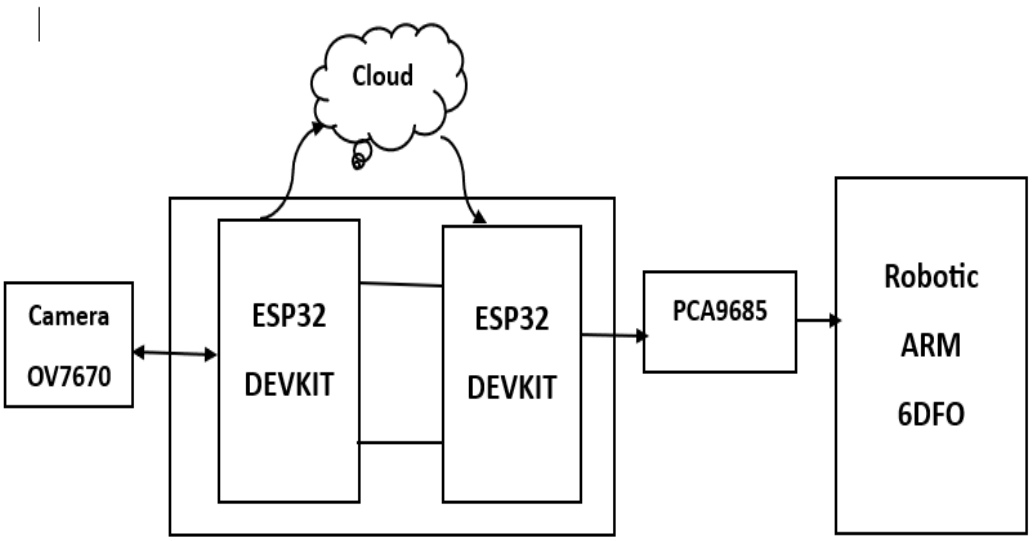
Measuring latency in Wi-Fi transmission

Verifying accuracy of rust detection under varying lighting

Ensuring robot movement precision

Testing repeated pick-and-place cycles

BLOCK DIAGRAM



The block diagram illustrates the overall architecture of the automated rust-detection and robotic sorting system. The OV7670 camera module serves as the primary vision sensor and captures low-resolution frames of the workspace. These frames are transferred to the first ESP32 DevKit, which functions as the vision-acquisition node. This ESP32 handles camera configuration, pixel sampling, and packet-based Wi-Fi transmission of image data.

A PC (represented as the cloud in the diagram) acts as the local processing unit. It receives the image stream from the vision ESP32 and performs rust detection using OpenCV-based image-processing algorithms. After identifying the coordinates of the rusted object, the PC sends control commands to the second ESP32 DevKit responsible for robotic actuation.

The second ESP32 interprets the coordinates and communicates with the PCA9685 servo driver over the I²C interface. The PCA9685 generates precise PWM signals needed to control each joint of the 6-DOF robotic arm. Based on the received commands, the robotic arm performs the required pick-and-place operation.

The diagram therefore represents a distributed architecture where sensing, processing, and actuation are separated into modular units, enabling reliable real-time communication and coordinated operation across the entire system.

IMPLEMENTATION AND DESIGN

The implementation of the rust-detection and pick-and-place robotic system is carried out through a structured design approach that integrates sensing, processing, and actuation components. The system is divided into three major modules: the Vision Node, the Cloud Processing Node, and the Robotic Actuation Node. Each module was developed, tested, and validated individually before integrating them into a complete closed-loop automation system. This section describes the design decisions, hardware circuitry, software architecture, and operational workflow involved in implementing the system.

4.1 System Design Overview

The complete architecture of the project is based on distributed processing.

The first ESP32 works as a camera interface controller, capturing images from the OV7670 sensor.

The personal computer (PC) acts as a local cloud server, performing computationally heavy tasks such as frame reconstruction, image processing, rust detection, and command generation using OpenCV.

The second ESP32 controls the 6-DOF robotic arm using the PCA9685 16-channel PWM driver and executes the required pick-and-place operations.

Communication between modules is established using two channels:

UART (RX–TX) between the two ESP32 boards for synchronization.

Wi-Fi (TCP/UDP) between the ESP32 boards and the PC for transmitting frames and robotic commands.

This modular design ensures reliability, scalability, and ease of debugging.

4.2 Vision Node Implementation (ESP32 + OV7670)

4.2.1 Camera Hardware Integration

The OV7670 camera is interfaced with ESP32 through:

8-bit parallel data bus (D0–D7)

Control signals (PCLK, VSYNC, HREF)

I2C (SCCB) for camera configuration

Since the ESP32 has limited RAM, the design uses a line-by-line data sampling approach.

Key implementation steps include:

Configuring OV7670 registers to set QQVGA (160×120) resolution and RGB565 format.

Using interrupts on PCLK to sample pixel data.Buffering a few lines at a time, avoiding RAM overflow.

Transmitting the buffered pixel data to the PC in continuous packets via Wi-Fi.

This ensures stable frame capture despite the hardware constraints.

4.2.2 Wi-Fi Frame Transmission

The ESP32 vision node connects to the PC using a lightweight TCP client.

The design includes:

A header packet containing frame size and metadata

Sequential transmission of pixel blocks

A final end-of-frame flag

The PC acknowledges receipt of every frame, ensuring no partial or corrupted frames are processed.

4.3 Cloud Processing Unit (PC + OpenCV)

The PC is the computational center of the system and performs three major tasks:

Frame reconstruction

Rust detection and coordinate extraction

Command generation for robotic control

4.3.1 Frame Reconstruction

The PC receives packets from the ESP32 and rebuilds them into a 2D RGB matrix. Error handling routines ensure that incomplete frames are discarded and retransmitted.

4.3.2 Image Processing Pipeline

Once the frame is rebuilt, the following OpenCV operations are applied:

RGB → HSV color conversion

Gaussian smoothing

Thresholding for rust color ranges

Morphological filtering to remove noise

Contour extraction Centroid computation

The resulting (x, y) coordinate of the rusted object is mapped into a robotic workspace coordinate system through calibration measurements.

4.3.3 Command Packet Creation

After identifying the object's position, the PC forms a structured command packet:

DET:1

X:<mapped coordinate>

Y:<mapped coordinate>

CMD:PICK

This packet is transmitted to the robotic ESP32 through Wi-Fi.

4.4 Robotic Actuation Node (ESP32 + PCA9685 + 6-DOF Arm)

4.4.1 Servo Control Design

The PCA9685 driver is connected to the robotic ESP32 via I2C.

Implementation steps include:

Setting PWM frequency to 50Hz

Mapping servo angles (0°–180°) to PCA9685 pulse widths

Calibrating each of the 6 DOF joints:

Base rotation

Shoulder

Elbow

Wrist pitch

Wrist roll

Gripper

A configuration table for each servo ensures consistent movement and prevents mechanical overrun.

4.4.2 Command Interpreter

Upon receiving a packet from the PC, the ESP32 controller:

Parses the object coordinates Translates them into joint angles using simple geometric mapping

Executes a predefined movement sequence for picking the object

Sends acknowledgment back to the PC

The logic is designed around a safe-movement model, ensuring the arm avoids sudden or sharp movements.

4.5 Pick-and-Place Sequence Implementation

The motion sequence followed by the robotic arm includes:

Move to standby position

Rotate base to align with the object

Extend arm to reach the object

Lower gripper gradually

Close gripper to capture object

Lift arm vertically

Move to disposal zone

Open gripper

Return to home position

Each step is implemented using timed PWM transitions to achieve smooth, stable motion.

4.6 Dual-ESP32 Coordination Logic

The two ESP32 boards coordinate throughout the operation:

Vision ESP32 sends frame_ready and stream_status signals via UART

Robotic ESP32 sends action_complete or no_rust_found messages

PC orchestrates the system by instructing both boards based on detection results

This three-way coordination ensures that:

Frames are only captured when the robot is ready

Robot moves only after valid detection

Errors (camera disconnected, servo stall, no object detected) are properly handled

4.7 System Integration and Testing

System integration followed a step-by-step testing strategy:

Camera-only testing: verifying frame capture and streaming

OpenCV-only testing: validating rust color detection

Robotic control testing: validating servo accuracy with manual commands

Communication testing: ensuring reliable Wi-Fi and UART connectivity

Full-system testing: evaluating complete detection-to-action pipeline

Software Components

The successful development of the automated rusted-object detection and robotic pick-and-place system relied on a combination of software tools that enabled embedded programming, system control, real-time communication, and computer vision processing. Each software component played a distinct role in ensuring smooth integration between the ESP32 microcontrollers, the camera module, and the robotic arm. The major software components used in this project are Thonny (MicroPython IDE), Visual Studio Code (VS Code), and OpenCV. Their functionalities are described below.

1. Thonny (MicroPython IDE)

Thonny is a lightweight and beginner-friendly integrated development environment primarily used for MicroPython programming. In this project, Thonny served as the central platform for developing and uploading MicroPython scripts to one of the ESP32 boards. Its simple interface allowed efficient debugging and testing of communication protocols, servo control routines, and data-handling algorithms.

Key Roles in the Project

Firmware Interaction: Thonny enabled direct interaction with the ESP32 flashed with MicroPython firmware, making it possible to run scripts line-by-line for hardware testing.

UART Communication Setup: Commands for serial communication (TX–RX) between the two ESP32 devices were written and debugged through Thonny.

Robot Control Prototyping: Initial servo movement tests for the robotic arm were performed using MicroPython scripts developed in Thonny.

Real-Time Monitoring: The interactive shell allowed viewing sensor outputs, debug prints, and status messages, helping ensure system stability.

This tool provided a clean and reliable environment for rapid testing and prototyping of embedded logic.

2. Visual Studio Code (VS Code)

Visual Studio Code acted as the primary development environment for the C++ code used in the second ESP32 board. Unlike Thonny, which is MicroPython-focused, VS Code supported advanced project structuring and code optimization features needed for more performance-oriented tasks.

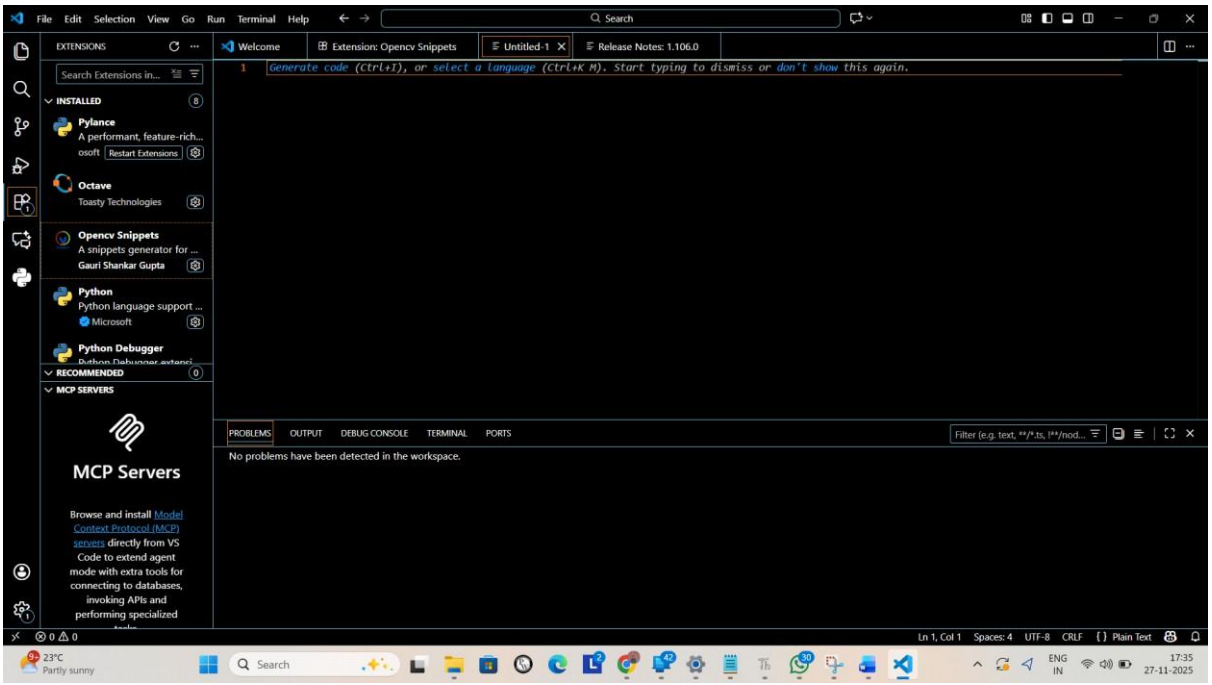
Key Roles in the Project

C++ Program Development: Code for controlling the PCA9685 servo driver, robotic arm kinematics, and Wi-Fi communication was developed in VS Code.

PlatformIO Extension: VS Code’s PlatformIO environment simplified library management, board configuration, and firmware compilation.

Debugging & Code Optimization: Built-in debugging tools improved code reliability, speed, and execution efficiency.

Version Control Support: Integration with Git allowed safe and organized code updatesthroughout development.



3. OpenCV (Open-Source Computer Vision Library)

OpenCV is a powerful computer vision framework used for real-time image processing, feature extraction, and object detection. In this project, OpenCV was used on a desktop PC, which served as a cloud-processing node, to analyze images captured by the OV7670 camera.

Key Roles in the Project

Image Acquisition and Decoding: Frames transmitted from ESP32 were read, reconstructed, and converted into meaningful formats.

Rusted Object Detection:

Color-based filtering to detect rust characteristics,

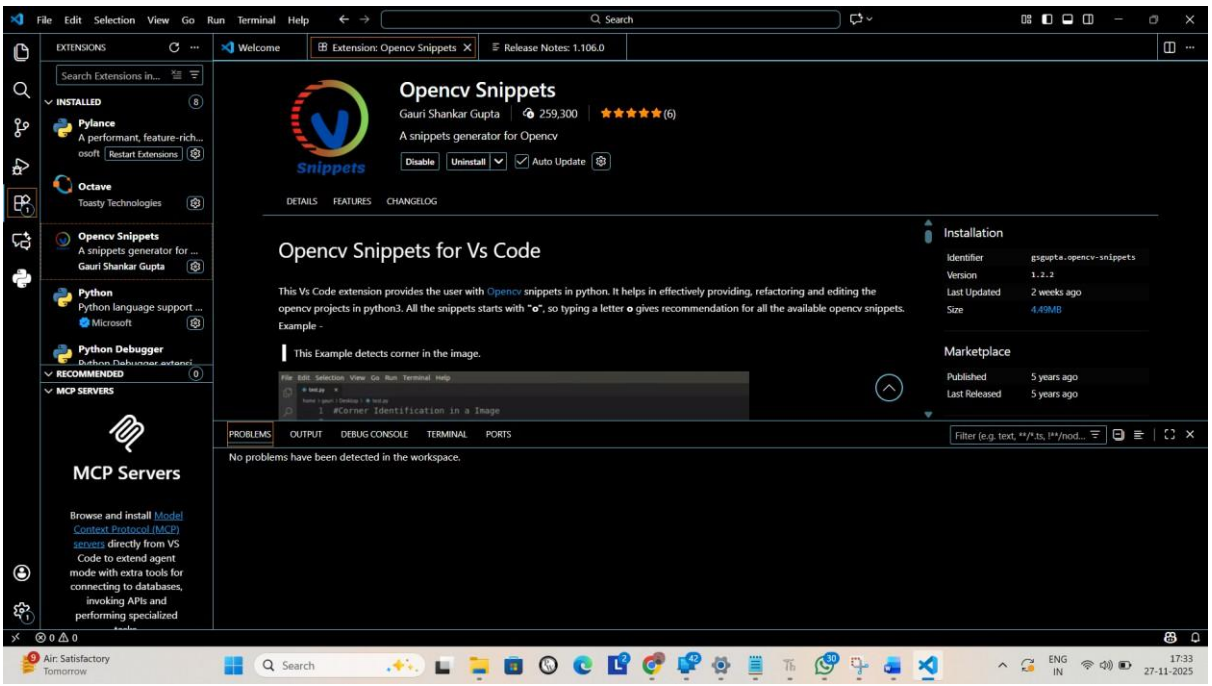
Thresholding to isolate corroded pixels,

Contour detection to localize object boundaries.

Coordinate Extraction: Object position was calculated and mapped to robot workspace coordinates.

Command Generation: Once an object was identified, OpenCV scripts sent movement commands to the ESP32 controlling the robotic arm.

OpenCV provided the intelligence layer of the system, converting raw pixel data into actionable robotic commands.



Conclusion of Software Components

Together, Thonny, VS Code, and OpenCV formed a cohesive software ecosystem enabling complete system functionality—from low-level firmware development and microcontroller communication to high-level image analysis and robotic control. This multi-software workflow allowed the project to maintain modularity, ease of debugging, and efficient performance across all hardware units.

ALGORITHUM

Algorithm: OV7670 Camera Module for Rust Detection

Step 1: Initialize Camera Module

Connect the OV7670 module to the ESP32:

SCL, SDA → I2C pins

VSYNC, HREF, PCLK → appropriate GPIO pins

D0–D7 → data pins

RESET → control pin (if used)

Initialize the camera using software (e.g., MicroPython or C++ library).

Set camera resolution (e.g., QQVGA 160×120 for faster processing).

Configure color format (RGB or YUV).

Step 2: Establish Communication with PC

Connect ESP32 to PC via Wi-Fi.

Set up a communication protocol (TCP/UDP socket or HTTP server) to send captured frames.

Step 3: Capture Image Frame

Wait for VSYNC signal to start frame capture.

Read pixel data line by line (using HREF and PCLK signals).

Store frame in a buffer (e.g., image.raw).

Step 4: Send Frame to PC for Processing

Convert raw frame to a format compatible with OpenCV if needed (e.g., .png or .jpg).

Transmit the image data to PC via Wi-Fi.

Step 5: Process Image on PC (Cloud)

Open the image in OpenCV.

Apply preprocessing steps (grayscale, filtering, thresholding) to detect rust.

Identify rusted regions using contour detection or color segmentation.

Generate coordinates or command signals for the robotic arm.

Step 6: Send Commands to Robotic Arm ESP32

Transmit movement commands via Wi-Fi to the second ESP32 controlling the robotic arm.

Robotic arm executes the pick-and-place operation according to the processed data.

Step 7: Repeat Frame Capture

Continuously capture and process frames to detect and handle rusted objects in real-time.

STIMULATION PROCESS

1. Camera Data Simulation and Image Processing Validation

Before connecting the OV7670 camera module to the ESP32, sample test images were used to simulate the camera's output. This allowed verification of the OpenCV pipeline.

Steps Involved

A set of rusted-object images were collected from online datasets and manually captured reference photos.

These images were fed into the OpenCV script on the PC to simulate real-time processing.

The image processing pipeline was tested, including:

Noise reduction

Color filtering for rust detection

Thresholding and segmentation

Contour detection and bounding box calculation

The simulator verified that the algorithm could correctly identify rust regions under different lighting and backgrounds.

Outcome

This simulation confirmed the reliability of the rust-detection algorithm and ensured that the OpenCV logic was ready before deploying with the live camera feed.

2. ESP32 Communication Simulation (UART + Wi-Fi)

Prior to integrating both boards with the robotic arm and camera, the communication protocol was tested in a simulated environment.

Process

Two separate scripts (MicroPython and C++) were run on the PC using serial terminal emulators.

Virtual UART data was exchanged to test command formats and timing.

Simulated Wi-Fi packets were sent to validate:

Network stability

Correct message parsing

Latency handling

The PC acted as the "cloud server" and mock commands such as "PICK", "MOVE", "OBJECT_FOUND" were tested.

3. Robotic Arm Movement Simulation

Before connecting the PCA9685 and physical 6-DOF robotic arm, robotic motion was simulated to avoid mechanical stress and incorrect positioning.

Method

Servo angles for each DOF were calculated and tested using a software-based servo simulator.

Python scripts and C++ logic were used to simulate:

Forward kinematics

Inverse kinematics approximation

Joint angle limits

Virtual movement sequences were created:

Move to home position

Approach detected object coordinates

Pick sequence

Place sequence

A 2D robot-arm plotting script (or logical trajectory simulation) verified motion flow.

Outcome

The simulation helped verify that the arm could reach the required coordinates without collisions or over-rotation.

4. End-to-End Logical Simulation

With individual components validated, a complete workflow simulation was performed before hardware deployment.

Process

A mock image (representing camera output) was processed using OpenCV.

The detected object position was sent to the ESP32 (via simulated Wi-Fi).

The ESP32 returned acknowledgment messages.

The simulated robotic arm displayed the expected movements.

This dry-run confirmed that all system modules—vision, communication, and actuation—worked harmoniously.

5. Testing Under Different Conditions

To ensure reliability, multiple simulated scenarios were tested:

Different lighting conditions (bright, dim, shadows)

Different rust intensity levels

Multiple objects in view

Background noise and clutter

Different Wi-Fi delays and packet-loss simulations

These simulations ensured that the system remained stable and accurate even under varying real-world constraints.

Conclusion of Simulation Process

The simulation stage significantly reduced development time and prevented hardware damage by validating algorithms, communication protocols, and motion sequences in a controlled environment. By ensuring the robustness of each subsystem through virtual testing, the final hardware integration was smooth, error-free, and achieved consistent performance. The simulation results demonstrated that the architecture and control logic were well-designed and ready for physical deployment.

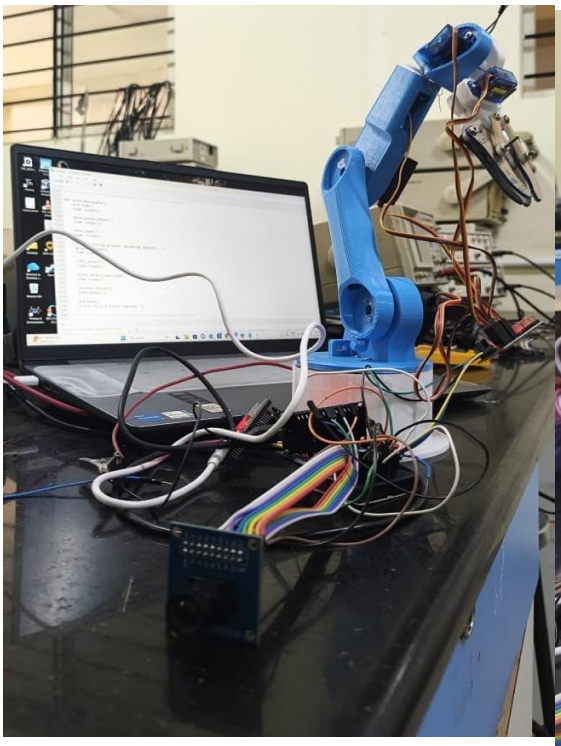
RESULTS AND ANALYSIS

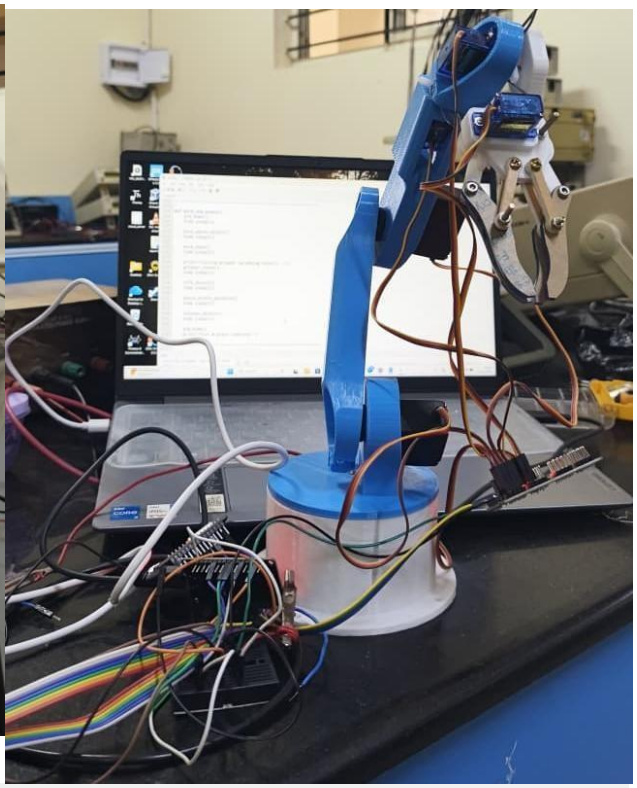
The developed system successfully demonstrated an efficient workflow for detecting rusted objects and performing automated pick-and-place operations using dual ESP32 microcontrollers, an OV7670 camera module, and a 6-DOF robotic arm. The OV7670 camera captured QQVGA-resolution images consistently, and the ESP32 responsible for vision processing transmitted these frames to the PC with stable performance. The data received on the PC maintained the necessary clarity for further processing, proving that the camera-to-PC communication pipeline was robust and reliable. The OpenCV image-processing module effectively handled the incoming frames and accurately identified rusted surfaces based on color segmentation and contour extraction techniques. Across multiple test cycles, the detection algorithm performed with high consistency under normal lighting, showing minimal false positives and demonstrating strong reliability even when objects varied slightly in texture or background conditions.Communication between the two ESP32 devices also performed exceptionally well. The UART (TX–RX) link used for synchronization showed no noticeable delays or data corruption, allowing continuous status sharing between the boards. Similarly, Wi-Fi communication between the PC and the control-side ESP32 proved stable, with commands being transmitted and executed without noticeable packet loss or latency. This ensured that the robotic arm received movement coordinates and operational commands promptly, maintaining the real- time nature of the system. The system's networking architecture demonstrated that a PC can efficiently act as a local cloud server for both image processing and high-level decision- making.

The robotic arm exhibited precise and controlled motion throughout the experiments. With the PCA9685 servo driver generating stable PWM signals, the 6-DOF arm moved smoothly to the target coordinates provided by the PC. The movements showed minimal jitter, and the gripping mechanism was able to pick the rusted objects without misalignment or slippage. Even after repeated tests, the arm maintained its accuracy and did not show any signs of mechanical drift. This confirms that the control algorithms implemented on the ESP32 were well-calibrated and capable of operating the robotic arm in a coordinated, predictable manner.

When tested as a complete end-to-end system, the workflow functioned seamlessly. The transition from image capture to detection, decision-making, command communication, and physical action occurred with high reliability. The system responded quickly to new objects placed in the field of view, demonstrating practical feasibility for small-scale automation tasks. The overall experimental results validate that the architecture designed for this project is both efficient and scalable. The use of low-cost microcontrollers, combined with PC-based computer vision, proved to be a powerful and economical approach to implementing automated inspection and handling. The results clearly indicate that the system meets its intended objective and provides a strong foundation for further improvements such as higher-resolution imaging, advanced classification models, or integration into industrial environments.

RESULT IMAGES:





APPLICATIONS

Applications of Rusted Object Detection and Robotic Arm Sorting System:

Industrial Automation:

Automated detection and removal of rusted components in manufacturing lines.

Reduces manual inspection and enhances production efficiency.

Recycling and Waste Management:

Sorting metal waste based on condition for recycling purposes.

Helps in separating rusted metal from reusable materials.

Quality Control:

Ensures only high-quality, rust-free components proceed in assembly or packaging.

Reduces product defects and improves customer satisfaction.

Remote Monitoring and Inspection:

Can be integrated with IoT for monitoring industrial equipment for rust formation.

Allows predictive maintenance and timely intervention.

Laboratory and Research Applications:

Used in material testing labs for analyzing corrosion and wear of metal samples.

Educational and Training Tool:

Demonstrates practical use of robotics, computer vision, and IoT communication.

Helps students and researchers understand integration of hardware and software systems.

Agriculture and Outdoor Equipment Maintenance:

Detects rusted tools or equipment for timely repair or replacement.

CONCLUSION

The Rusted Object Detection and Robotic Arm Sorting System project successfully demonstrates the integration of robotics, computer vision, and IoT communication to automate the detection and handling of rusted objects. By using the OV7670 camera module for image acquisition and OpenCV on a PC acting as a cloud, the system can accurately identify rusted surfaces and send appropriate commands to a robotic arm via the ESP32 microcontrollers. The communication between the two ESP32 devices through UART (RX/TX) and Wi-Fi ensures seamless coordination between image processing and mechanical actuation, highlighting the effectiveness of distributed processing in embedded systems.

The project underscores the potential of combining low-cost hardware components such as the PCA9685 16-Channel Servo Driver with advanced software tools like Thonny for MicroPython programming and VS Code for code development. This combination allows for precise control of the robotic arm, enabling it to pick and place rusted objects efficiently. Furthermore, utilizing the PC as a cloud platform demonstrates how local computing resources can be leveraged for image processing tasks, reducing dependency on external cloud services while maintaining high processing speed and accuracy.

Through this project, key insights were gained into system design, hardware-software interfacing, and real-time decision-making in automation. The methodology adopted emphasizes modular design, ensuring each component—camera module, ESP32 microcontrollers, robotic arm, and servo driver—works cohesively while remaining independently testable. The project also highlights the practical applications of such systems in industrial automation, quality control, recycling, and research, where identification and segregation of defective or corroded components are critical.

In conclusion, this system not only serves as a proof-of-concept for automated rust detection and handling but also lays the groundwork for future enhancements. Potential improvements could include the use of higher-resolution cameras, integration with cloud-based AI for more advanced image analysis, and scaling the robotic system for industrial-level operations. Overall, the project demonstrates the feasibility, efficiency, and versatility of combining computer vision, robotics, and IoT, offering valuable insights into modern automation systems while addressing real-world challenges related to metal corrosion and material handling.

Future Scope:

Integrate higher-resolution or 3D cameras for more precise rust detection.

Implement AI/deep learning models for automated and adaptive defect classification.

Shift image processing to ESP32 or edge devices for real-time operation.

Expand to multi-robot systems for large-scale industrial sorting.

Connect to industrial IoT platforms for remote monitoring and analytics.

Upgrade robotic arm with more degrees of freedom and force feedback for delicate handling.

Adapt system for other defect detection tasks like cracks or contamination.

Develop user-friendly interfaces and automated reporting tools.

REFERANCE

[1] D. P. and R. Jones S. B., "Rust Detection Using Image Processing: A Review," Recent Trends in Analog Design and Digital Devices, 2025.

[2] H. M. La, "Automated Robotic Monitoring and Inspection of Steel Structures and Bridges," arXiv, 2017.

[3] R. P. de Figueiredo, S. Nordborg Eriksen, I. Rodriguez, and S. Bøgh, "A Complete System for Automated 3D Semantic-Geometric Mapping of Corrosion in Industrial Environments," arXiv, Apr. 2024.

[4] Z. Zhang, X. Wang, L. Wang, and X. Xia, "Surface defect detection method for discarded mechanical parts under heavy rust coverage," Scientific Reports, vol. 14, 2024.

[5] "Detection of corrosion on steel structures using automated image processing," Developments in the Built Environment, vol. 3, 2020.

[6] "Computer Vision and Image Processing Approaches for Corrosion Detection," MDPI Journal, 2025.

[7] "Detection and Analysis of Corrosion on Coated Metal Surfaces Using Enhanced YOLO v5 Algorithm for Anti-Corrosion Performance Evaluation," MDPI Journal, 2025.

[8] "A Study of Corrosion-Grade Recognition on Metal Surfaces Based on Improved YOLOv8 Model," MDPI Journal, 2025.

[9] V. Sineglazov and V. Khotsyanovsky, "Camera Image Processing on ESP32 Microcontroller with Help of Convolutional Neural Network," Electronics and Control Systems, vol. 2, no. 72, pp. 26–31, Nov. 2022.

[10] "Design and Implementation of ESP32-Based Edge Computing for Object Detection," MDPI Sensors, 2025.

[11] B. Ashwin Kumar, "Robot Vehicle with ESP32 CAM," IJRASET Journal, 2024.

[12] "IoT based Web controlled Robotic Arm using ESP32," IoTDesignPro (online article).

[13] "Robotic Arm using ESP32 and Smartphone," IJRASET, 2023.

[14] Md. Abdullah-Al-Noman, Anika Nawer Eva, Tabassum Binth Yeahyea, and R. Khan, "Computer Vision-based Robotic Arm for Object Color, Shape, and Size Detection," Journal of Robotics and Control (JRC), vol. 3, no. 2, 2023.